

# République du Sénégal



MINISTÈRE DE L'ENSEIGNEMENT SUPÉRIEUR ET DE LA RECHERCHE

DIRECTION GÉNÉRALE DE L'ENSEIGNEMENT SUPÉRIEUR



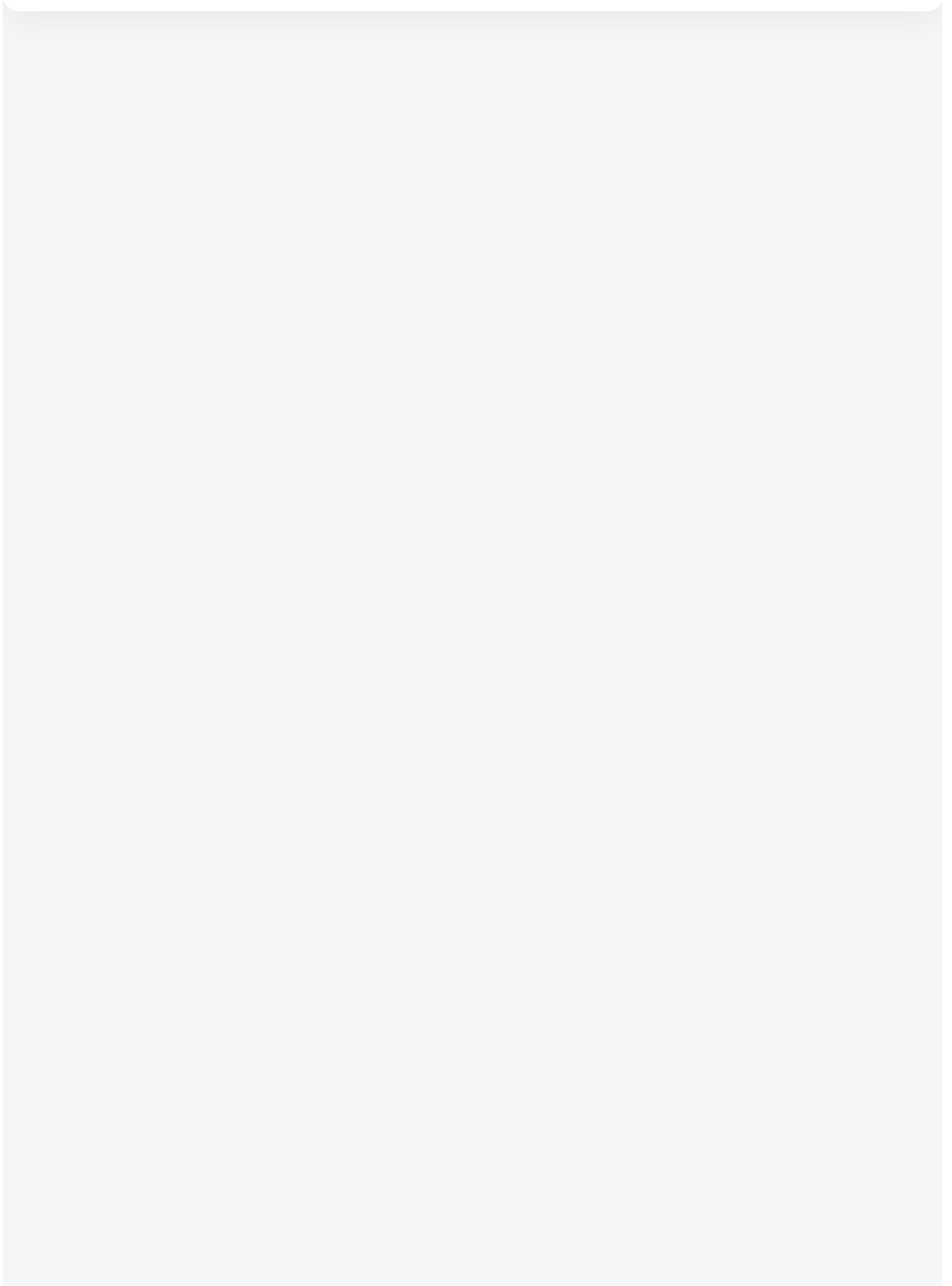
## Rapport AWS IAM — Gestion des accès et sécurité

Présenté par : **Ibrahima Khalilou Thiam**

Master 2 Génie Logiciel UDB

Matricule 003

À **M. Massamba LO**



# Sommaire

1. [Objectifs et contexte](#)
2. [Prérequis](#)
3. [Concepts et architecture](#)
4. [Installation et configuration](#)
5. [Réalisation pratique](#)
6. [Commandes et explications](#)
7. [Vérifications et tests](#)
8. [Dépannage](#)
9. [Sécurité et bonnes pratiques](#)
10. [Checklist](#)
11. [Conclusion](#)

# 1. Objectifs et contexte

AWS Identity and Access Management (**IAM**) permet de contrôler l'authentification et l'autorisation d'accès aux ressources AWS. L'objectif du TP est de construire une gestion des identités conforme au principe du moindre privilège.

- Comprendre utilisateurs, groupes, rôles et politiques IAM ;
- sécuriser le compte racine avec la MFA ;
- créer des identités et leur attribuer uniquement les permissions nécessaires ;
- utiliser des rôles IAM pour les services AWS ;
- auditer et vérifier les permissions avec l'AWS CLI.

**Principe fondamental** : une permission doit être accordée uniquement lorsqu'elle est nécessaire, sur la ressource la plus précisément possible.

## 2. Prérequis

- Compte AWS actif ;
- accès à la console AWS ;
- AWS CLI v2 installée si la partie CLI est réalisée ;
- application ou dispositif compatible MFA ;
- connaissances de base en JSON et en administration cloud.

**Important :** éviter de créer des clés d'accès longues durées lorsque l'authentification fédérée, IAM Identity Center ou un rôle IAM permet de répondre au besoin.

### 3. Concepts et architecture

| Composant | Rôle                                                                                    |
|-----------|-----------------------------------------------------------------------------------------|
| User      | Identité IAM représentant une personne ou un besoin spécifique.                         |
| Group     | Ensemble d'utilisateurs auquel des politiques peuvent être associées.                   |
| Role      | Identité assumable fournissant des permissions temporaires, notamment aux services AWS. |
| Policy    | Document JSON décrivant les actions autorisées ou refusées.                             |
| ARN       | Identifiant permettant de cibler précisément une ressource AWS.                         |
| MFA       | Second facteur renforçant l'authentification.                                           |

Principal → Authentification → IAM Policy → Autorisation → Ressource AWS Utilisateur  
→ Groupe → Policy → S3 / EC2 / autres services Service AWS → Role IAM →  
Policy → Ressource AWS

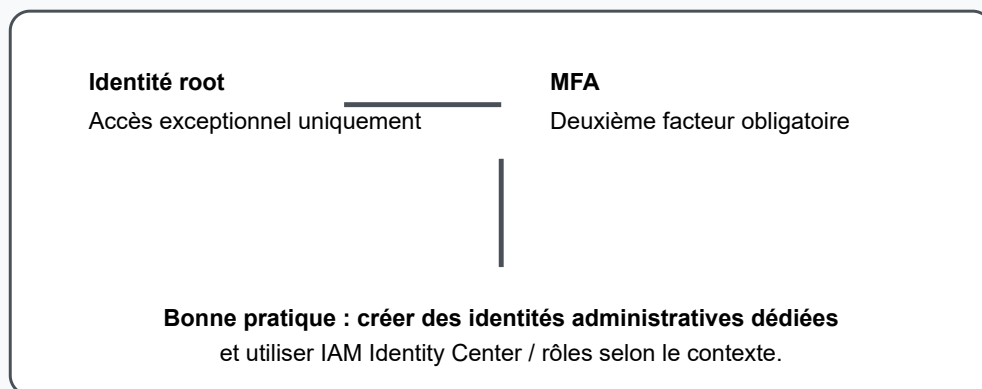
**Différence importante :** l'identité qui demande l'accès est le principal ; la policy détermine les permissions ; une ressource AWS peut également posséder une policy basée sur la ressource.

## 4. Installation et configuration

### 4.1 Sécuriser le compte racine

1. Utiliser le compte racine uniquement pour les opérations qui l'exigent.
2. Activer la MFA sur le compte racine.
3. Ne pas utiliser ses identifiants pour les tâches quotidiennes.
4. Vérifier et supprimer les clés d'accès racine inutiles.

#### Protection du compte root AWS par MFA



### 4.2 Préparer l'administration

Pour un environnement professionnel, privilégier AWS IAM Identity Center ou une fédération d'identité pour les utilisateurs humains. Les rôles IAM sont privilégiés pour les workloads AWS.

### 4.3 Vérifier l'identité CLI

```
aws sts get-caller-identity aws configure list
```

La première commande identifie le principal utilisé par la session. La seconde affiche la provenance de la configuration sans révéler le secret d'accès.

## 5. Réalisation pratique

### 5.1 Création d'un groupe

```
aws iam create-group --group-name Developers aws iam list-groups
```

### 5.2 Création d'une politique limitée à S3

```
{ "Version": "2012-10-17", "Statement": [{ "Effect": "Allow", "Action": ["s3:ListBucket"], "Resource": "arn:aws:s3:::mon-bucket-s3" }, { "Effect": "Allow", "Action": ["s3:GetObject"], "Resource": "arn:aws:s3:::mon-bucket-s3/*" }] }
```

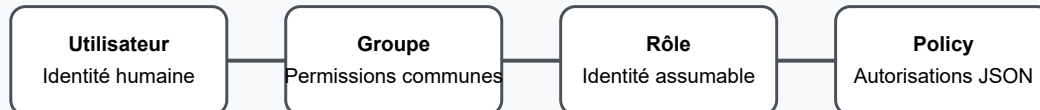
Enregistrer ce document dans `politique-s3.json`, puis :

```
aws iam create-policy \ --policy-name S3LectureMonBucket \ --policy-document file://politique-s3.json
```

### 5.3 Créer un utilisateur et l'ajouter au groupe

```
aws iam create-user --user-name dev-moussa aws iam add-user-to-group \ --group-name Developers \ --user-name dev-moussa
```

#### Modèle IAM : utilisateurs, groupes, rôles et politiques



#### Principe du moindre privilège

Accorder uniquement les actions et ressources nécessaires.  
Préférer les rôles et identifiants temporaires pour les workloads.

### 5.4 Préférer un rôle pour EC2

Une instance EC2 doit recevoir un rôle IAM via un instance profile lorsqu'elle doit appeler AWS. Cela évite de placer une clé d'accès statique dans le serveur.

## 6. Commandes et explications

| Commande                                         | Fonction                                                  |
|--------------------------------------------------|-----------------------------------------------------------|
| <code>aws sts get-caller-identity</code>         | Identifie le principal de la session courante.            |
| <code>aws iam list-users</code>                  | Liste les utilisateurs IAM.                               |
| <code>aws iam list-groups</code>                 | Liste les groupes.                                        |
| <code>aws iam list-attached-user-policies</code> | Affiche les politiques gérées attachées à un utilisateur. |
| <code>aws iam list-group-policies</code>         | Liste les politiques inline d'un groupe.                  |
| <code>aws iam get-policy</code>                  | Récupère les métadonnées d'une policy gérée.              |
| <code>aws iam simulate-principal-policy</code>   | Évalue si des actions sont autorisées pour un principal.  |
| <code>aws iam list-access-keys</code>            | Inventorie les clés d'accès d'un utilisateur.             |

```
aws iam list-attached-user-policies --user-name dev-moussa aws iam list-groups-for-user --user-name dev-moussa aws iam list-access-keys --user-name dev-moussa
```

## 7. Vérifications et tests

### 7.1 Vérifier l'identité

```
aws sts get-caller-identity
```

### 7.2 Vérifier l'appartenance au groupe

```
aws iam list-groups-for-user --user-name dev-moussa
```

### 7.3 Vérifier les permissions

```
aws iam simulate-principal-policy \ --policy-source-arn  
arn:aws:iam::123456789012:user/dev-moussa \ --action-names s3:ListBucket \ --  
resource-arns arn:aws:s3:::mon-bucket-s3
```

Le résultat doit être interprété à partir de `EvalDecision`. Une permission refusée doit être analysée avant toute modification de policy.

### 7.4 Audit

```
aws iam get-account-summary aws iam generate-credential-report aws iam get-  
credential-report
```

## 8. Dépannage

| Problème             | Diagnostic                                                | Action                                                        |
|----------------------|-----------------------------------------------------------|---------------------------------------------------------------|
| AccessDenied         | Vérifier politiques identité, ressource et SCP éventuels. | Identifier la permission exacte avant de modifier les droits. |
| Mauvais principal    | <code>aws sts get-caller-identity</code>                  | Corriger le profil ou le rôle utilisé.                        |
| Policy inefficace    | Vérifier ARN, Action, Resource et conditions.             | Tester avec <code>simulate-principal-policy</code> .          |
| Clé compromise       | Inventorier les clés et leur dernière utilisation.        | Désactiver puis supprimer la clé après rotation.              |
| EC2 sans permissions | Vérifier le rôle et l'instance profile.                   | Associer le rôle IAM approprié à l'instance.                  |

**Attention :** ne pas résoudre systématiquement un `AccessDenied` en donnant `AdministratorAccess`. Il faut d'abord déterminer l'action et la ressource réellement nécessaires.

## 9. Sécurité et bonnes pratiques

- Protéger le compte racine avec MFA et ne pas l'utiliser quotidiennement.
- Privilégier IAM Identity Center/fédération pour les utilisateurs humains.
- Utiliser des rôles et des identifiants temporaires pour les workloads.
- Appliquer le moindre privilège et cibler les ressources par ARN.
- Éviter les politiques avec **Action: "\*"** et **Resource: "\*"** lorsqu'elles ne sont pas indispensables.
- Surveiller les clés d'accès et supprimer les identifiants inutilisés.
- Utiliser CloudTrail et IAM Access Analyzer pour l'audit et l'analyse des accès.
- Ne jamais placer de secrets dans Git, des scripts ou des AML.
- Utiliser des politiques de contrôle organisationnel lorsque AWS Organizations est disponible.

**Principe de défense en profondeur :** IAM ne doit pas être considéré isolément. Il complète les contrôles réseau, le chiffrement, la journalisation, la supervision et la gestion des vulnérabilités.

## 10. Checklist

- ☐ MFA activée sur le compte racine
- ☐ Compte racine réservé aux tâches exceptionnelles
- ☐ Identité d'administration correctement configurée
- ☐ Groupes et rôles définis selon les responsabilités
- ☐ Politiques limitées au besoin réel
- ☐ Utilisateurs humains gérés avec une méthode d'identité adaptée
- ☐ Rôles IAM utilisés pour EC2/Lambda lorsque nécessaire
- ☐ Aucune clé secrète stockée dans le code
- ☐ Clés d'accès inutiles supprimées ou désactivées
- ☐ Permissions testées avec les outils IAM
- ☐ CloudTrail et mécanismes d'audit prévus
- ☐ IAM Access Analyzer utilisé pour identifier les accès externes

# 11. Conclusion

Ce travail pratique a permis d'étudier AWS IAM comme composant central de la sécurité d'une infrastructure cloud. Les notions d'utilisateur, groupe, rôle, policy et MFA permettent de structurer les accès tout en réduisant la surface de risque.

La mise en œuvre montre également l'importance de privilégier les rôles et les identifiants temporaires, de tester les permissions et de conserver une traçabilité des opérations. Une architecture IAM correctement conçue constitue ainsi une base indispensable pour sécuriser les services AWS.

Dans un environnement professionnel, cette démarche doit être complétée par la fédération d'identité, l'audit continu, l'analyse des accès, la rotation des secrets et une gouvernance centralisée.

